

AWS FOR DEVOPS INTERNS

Amazon Web Services Core Services Explained

EC2 • VPC • S3 • IAM • RDS • **EKS** & more

Detailed concepts • Architecture diagrams

+ Top 50 Interview Questions & Answers

A beginner-friendly but technically complete guide,
with special focus on Amazon EKS (Kubernetes) architecture.

What's Inside

Part 1 — Concepts & Architecture

1. Global Infrastructure (Regions, AZs, Edge)
2. EC2 — virtual servers, AMIs, storage, security
3. VPC — private networking, subnets, gateways (with diagram)
4. S3 — object storage, classes, security
5. IAM — users, roles, policies, least privilege
6. RDS — managed databases, Multi-AZ, replicas
7. **EKS — managed Kubernetes (special deep-dive + architecture diagram)**
8. DevOps on AWS — CI/CD, IaC, monitoring (with diagram)

Part 2 — Top 50 Interview Questions & Answers

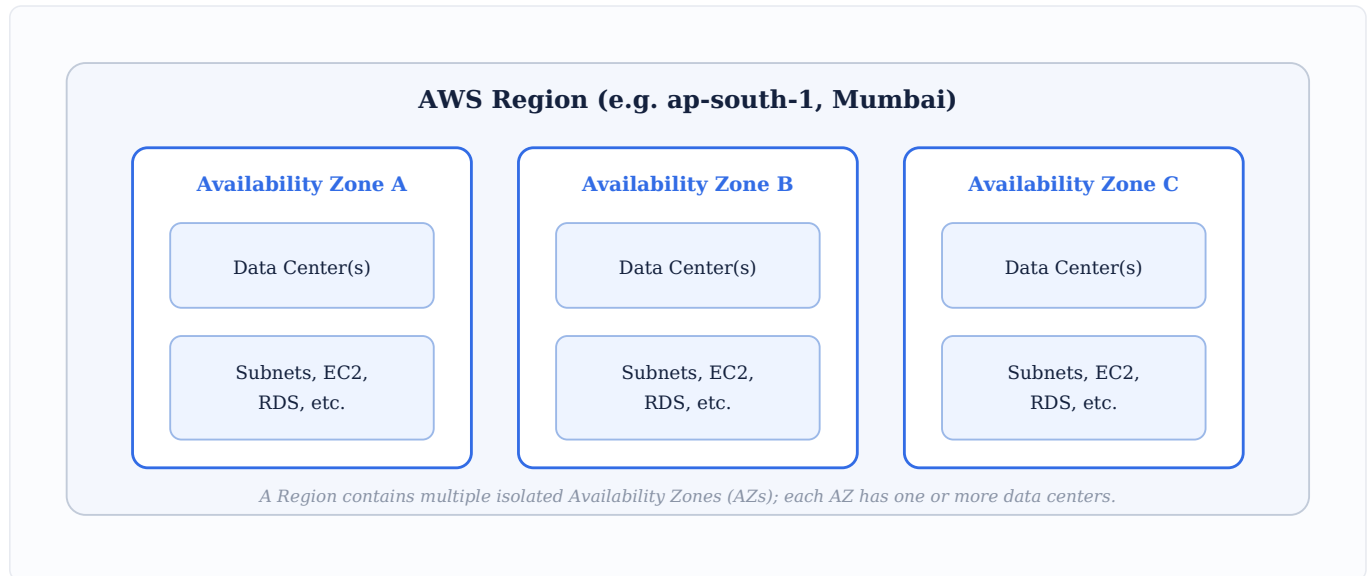
Grouped by service, written simply with the technical terms interviewers expect.

Part 1 — Concepts & Architecture

AWS (Amazon Web Services) is the world's most widely used cloud platform. Instead of buying physical servers, you rent computing, storage, networking, and databases **on demand** and pay only for what you use. For a DevOps engineer, AWS is the foundation on which applications are built, deployed, scaled, and monitored.

1. AWS Global Infrastructure

AWS is organized into **Regions** and **Availability Zones (AZs)**. Understanding this is essential for designing **highly available** systems.



- **Region** — a geographic area (e.g. `ap-south-1` Mumbai). You choose Regions based on latency, cost, and compliance.
- **Availability Zone (AZ)** — one or more isolated data centers within a Region. Spreading resources across AZs gives **fault tolerance**.
- **Edge Location** — caching sites used by CloudFront (CDN) to deliver content close to users with low latency.

2. EC2 — Elastic Compute Cloud

EC2 provides resizable **virtual servers** called **instances**. It is the most fundamental compute service in AWS.

Key building blocks

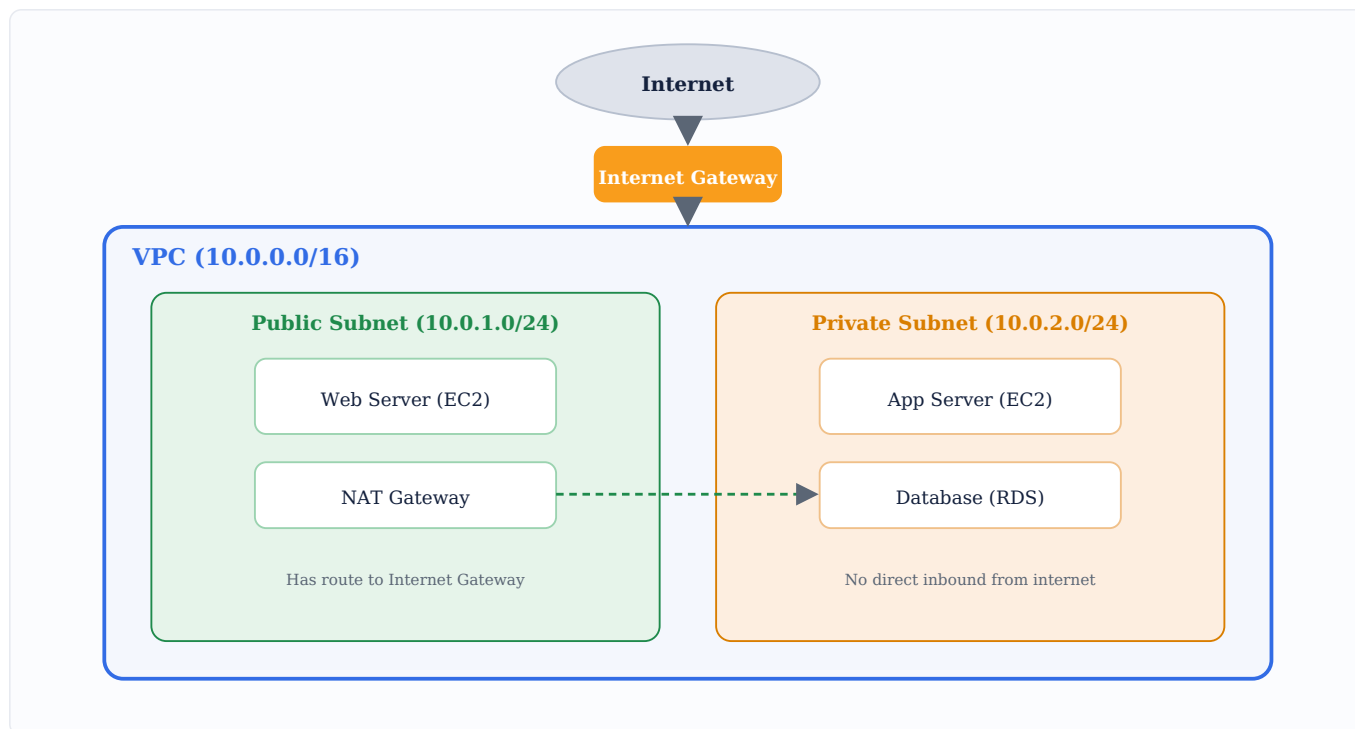
- **AMI (Amazon Machine Image)** — the OS + software template used to launch an instance.
- **Instance type** — the hardware profile (vCPU, memory), e.g. `t3.micro`.
- **EBS (Elastic Block Store)** — persistent disk volumes attached to instances; back up via **snapshots**.
- **Security Group** — a stateful virtual firewall controlling inbound/outbound traffic.
- **Key pair** — public/private keys used for secure SSH/RDP login.

Purchasing options & scaling

Pay with **On-Demand** (flexible), **Reserved/Savings Plans** (discount for commitment), or **Spot** (cheapest, but interruptible). **Auto Scaling** automatically adds/removes instances to match demand, and an **Elastic Load Balancer (ELB)** distributes traffic across them.

3. VPC — Virtual Private Cloud

A **VPC** is your own **isolated virtual network** inside AWS. You control the IP range, subnets, routing, and what is exposed to the internet.



- **Subnet** — a range of IPs within the VPC, tied to one AZ. **Public subnets** reach the internet (via an Internet Gateway); **private subnets** do not.
- **Internet Gateway (IGW)** — connects the VPC to the internet.
- **NAT Gateway** — lets private instances make **outbound-only** internet connections safely.
- **Route Table** — rules that direct network traffic.
- **Security Group (instance-level, stateful)** vs **NACL (subnet-level, stateless)**.

Common design: put web servers in a public subnet and databases (RDS) in a private subnet, so the database is never directly exposed to the internet.

4. S3 — Simple Storage Service

S3 is highly durable **object storage**. You store **objects** (files plus metadata) inside **buckets**. It offers **99.999999999% (11 nines) durability** and effectively unlimited scale.

- **Storage classes** — Standard (frequent), Standard-IA / One Zone-IA (infrequent), Intelligent-Tiering (auto), Glacier / Deep Archive (archival).
- **Versioning** — keep multiple versions to recover from accidental deletes/overwrites.
- **Lifecycle policies** — auto-transition or expire objects to cut cost.
- **Security** — bucket policies, Block Public Access, encryption at rest & in transit.

Common uses: backups, static website hosting, data lakes, and storing container/build artifacts.

5. IAM — Identity & Access Management

IAM controls **who can do what** in your AWS account. It is the backbone of AWS security.

- **User** — a permanent identity for a person/app (long-term credentials).
- **Group** — a collection of users sharing permissions.
- **Role** — a temporary identity that is **assumed** (ideal for EC2, services, cross-account).
- **Policy** — a JSON document defining allowed/denied **actions** on **resources**.

Best practices: follow **least privilege**, enable **MFA** (especially on the root account), use **roles instead of hard-coded keys**, and avoid using the root account for daily work.

6. RDS — Relational Database Service

RDS is a **managed relational database** — AWS handles patching, backups, and scaling so you focus on your data. Supported engines include **MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Aurora**.

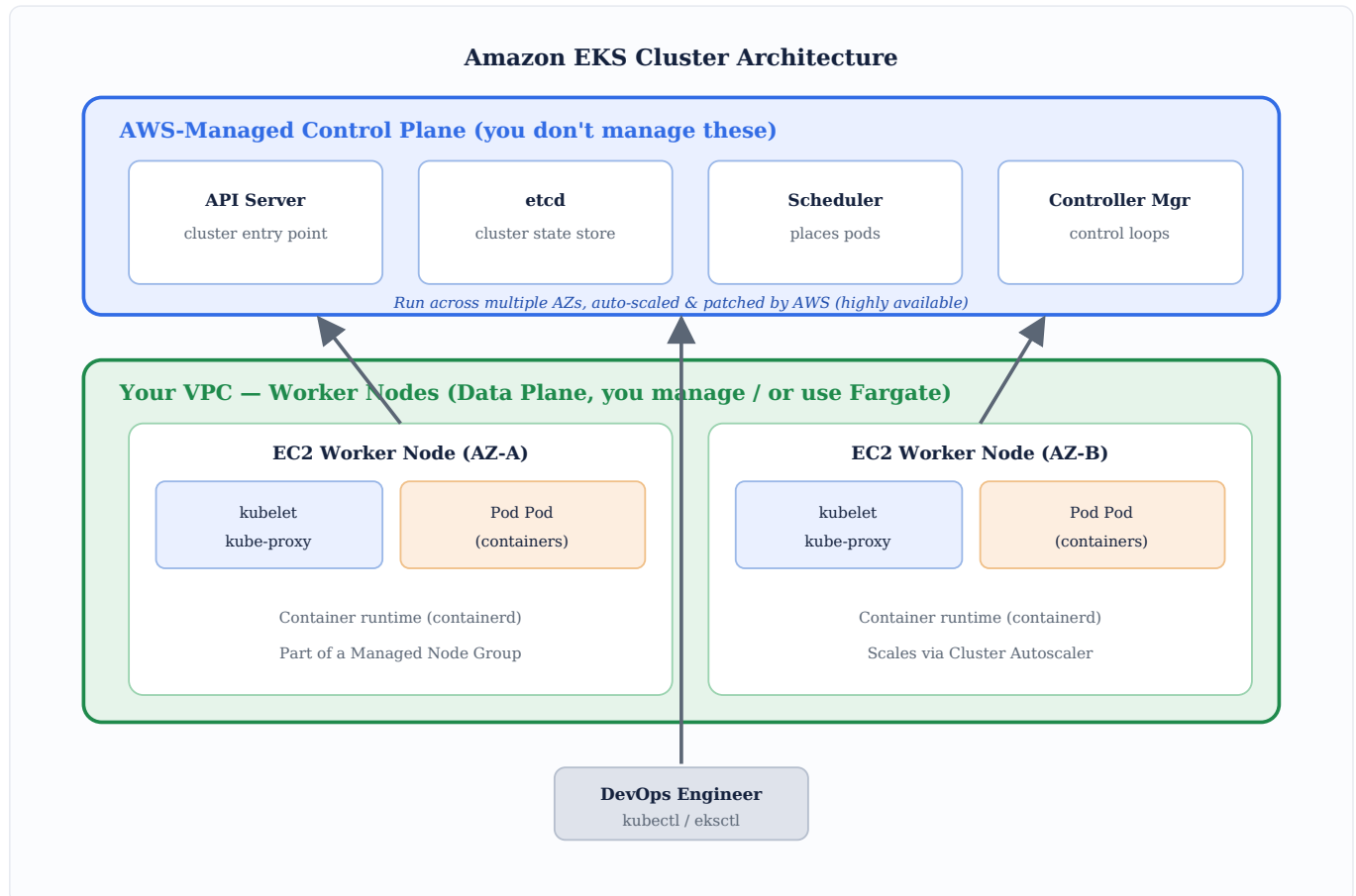
- **Multi-AZ** — a synchronous standby in another AZ for **automatic failover** / **high availability**.
- **Read Replicas** — asynchronous copies to **scale read traffic**.
- **Aurora** — AWS's high-performance, cloud-native engine compatible with MySQL/PostgreSQL.
- For **NoSQL** needs at massive scale, use **DynamoDB** instead.

7. EKS — Elastic Kubernetes Service SPECIAL DEEP-DIVE

For a DevOps engineer, **EKS** is one of the most important AWS services because it runs **Kubernetes** — the industry-standard container orchestrator — as a **managed service**. EKS removes the hardest part of running Kubernetes: operating the **control plane**.

The key idea: AWS manages the brain, you manage the workers

In a self-managed Kubernetes cluster you must install, secure, scale, and patch the control plane yourself — which is complex and error-prone. With **EKS**, **AWS runs the entire control plane for you**, across multiple Availability Zones, fully patched and highly available. You only focus on your **worker nodes** and your **applications**.



The two planes of EKS

- **Control Plane (AWS-managed)** — the **API Server** (cluster entry point), **etcd** (stores all cluster state), the **Scheduler** (decides which node runs each Pod), and the **Controller Manager** (runs control loops). AWS runs these across multiple AZs and you are billed a flat hourly fee per cluster.
- **Data Plane (your responsibility)** — the **worker nodes** that actually run your containers. These can be **EC2 instances** (managed via **Managed Node Groups**) or **AWS Fargate** (serverless — no nodes to manage at all).

How worker nodes connect

Each worker node runs the **kubelet** (the node agent that talks to the API Server) and **kube-proxy** (network rules), plus a **container runtime (containerd)**. EKS uses the **Amazon VPC CNI plugin** so that each Pod receives a real **VPC IP address**, and **IAM Roles for Service Accounts (IRSA)** give Pods fine-grained AWS permissions.

EKS vs. ECS vs. Fargate

- **EKS** — managed, standard **Kubernetes** (portable, large ecosystem).
- **ECS** — AWS's **proprietary** orchestrator (simpler, AWS-native).
- **Fargate** — **serverless compute** for containers; a launch type that runs Pods/tasks **without managing EC2 nodes** (works with both EKS and ECS).

Deploying to EKS (typical DevOps flow)

Build a Docker image → push it to **Amazon ECR** (the managed container registry) → apply Kubernetes manifests (Deployments, Services) with **kubectl** or **Helm** → often automated through a CI/CD pipeline. The cluster itself is commonly

created with **eksctl** or Terraform.

Why interns should know EKS well: it sits at the intersection of containers, Kubernetes, networking (VPC), security (IAM), and CI/CD — exactly the skills a DevOps role exercises daily.

8. DevOps on AWS — CI/CD, IaC & Monitoring

AWS provides native tools to automate the software delivery lifecycle.

A typical AWS-native CI/CD pipeline (CodePipeline orchestrates these)



Source → Build → Deploy → Run on container service → Monitor

- **CodeCommit** (Git repo) → **CodeBuild** (build/test) → **CodeDeploy** (deploy) → orchestrated by **CodePipeline**.
- **CloudFormation** — Infrastructure as Code (define resources in YAML/JSON); **Terraform** is a popular multi-cloud alternative.
- **CloudWatch** — metrics, logs, dashboards, and alarms for monitoring and automated responses.
- **ECR** — managed registry for container images used by EKS/ECS.

Top 50 AWS Interview Questions & Answers

Commonly asked of DevOps engineer interns, grouped by service. Answers are concise but keep the technical terms you're expected to use; **bolded** words are the keywords worth remembering.

General & Global Infrastructure

Q1. What is AWS and what are its main service categories?

Answer: **AWS (Amazon Web Services)** is a cloud computing platform offering **on-demand** IT resources over the internet on a **pay-as-you-go** basis. Main categories include **Compute** (EC2, Lambda, EKS), **Storage** (S3, EBS), **Networking** (VPC, Route 53), **Databases** (RDS, DynamoDB), and **Security/Identity** (IAM).

Q2. What is the difference between a Region and an Availability Zone (AZ)?

Answer: A **Region** is a separate **geographic area** (e.g. Mumbai, ap-south-1). An **Availability Zone** is one or more **isolated data centers** within a Region, with independent power and networking. You deploy across **multiple AZs** for **high availability and fault tolerance**.

Q3. What is an Edge Location?

Answer: An **Edge Location** is a site used by **CloudFront** (AWS's CDN) to **cache content closer to users** for low-latency delivery. There are many more edge locations than Regions, and they are also used by services like Route 53 and AWS Global Accelerator.

Q4. What does the AWS Shared Responsibility Model mean?

Answer: It splits security duties: AWS is responsible for **security OF the cloud** (hardware, the global infrastructure, managed services), while the customer is responsible for **security IN the cloud** (data, IAM configuration, OS patching on EC2, security groups). The exact split shifts toward AWS for more managed services.

Q5. What is the difference between scalability and elasticity in AWS?

Answer: **Scalability** is the ability to **handle growth** by adding resources (vertically by resizing, or horizontally by adding instances). **Elasticity** is the ability to **automatically scale resources up and down** in response to real-time demand (e.g. Auto Scaling), so you pay only for what you need.

EC2 — Elastic Compute Cloud

Q6. What is Amazon EC2?

Answer: **EC2 (Elastic Compute Cloud)** provides resizable **virtual servers (instances)** in the cloud. You choose an **instance type** (CPU/memory), an **AMI** (the OS image), storage, and networking, then launch and manage the server as needed.

Q7. What is an AMI?

Answer: An **AMI (Amazon Machine Image)** is a **template** containing the operating system, application server, and applications used to **launch an EC2 instance**. You can use AWS-provided, Marketplace, or custom AMIs for consistent, repeatable deployments.

Q8. What are the different EC2 instance purchasing options?

Answer: **On-Demand** (pay per second/hour, no commitment), **Reserved Instances / Savings Plans** (1-3 year commitment for big discounts), **Spot Instances** (spare capacity at steep discounts but can be reclaimed), and **Dedicated Hosts** (physical servers for compliance/licensing).

Q9. What is the difference between stopping and terminating an EC2 instance?

Answer: **Stopping** shuts down the instance but **keeps the EBS root volume and data**; you can start it again later (you still pay for storage). **Terminating permanently deletes** the instance and, by default, its root volume — it cannot be restarted.

Q10. What is a Security Group?

Answer: A **Security Group** is a **virtual firewall** attached to an instance's network interface that controls **inbound and outbound traffic** via rules. It is **stateful** — return traffic for an allowed request is automatically permitted. You specify allow rules only (no deny rules).

Q11. What is the difference between a Security Group and a Network ACL (NACL)?

Answer: A **Security Group** operates at the **instance level**, is **stateful**, and supports allow rules only. A **NACL** operates at the **subnet level**, is **stateless** (you must allow return traffic explicitly), and supports both **allow and deny** rules.

Q12. What is Amazon EBS?

Answer: **EBS (Elastic Block Store)** provides **persistent block storage volumes** for EC2 instances — like a virtual hard disk. Data persists independently of the instance lifecycle, volumes are **AZ-specific**, and you can take **snapshots** (backups) stored in S3.

Q13. What is an Elastic IP address?

Answer: An **Elastic IP (EIP)** is a **static, public IPv4 address** you can allocate to your account and **remap to any instance**. It is useful for keeping a fixed public address even if the underlying instance is replaced.

Q14. What is EC2 Auto Scaling and what is a launch template?

Answer: **Auto Scaling** automatically **adds or removes EC2 instances** based on demand/metrics or a schedule, keeping capacity matched to load. A **launch template** defines the instance configuration (AMI, type, security groups) that the Auto Scaling group uses to launch new instances.

VPC — Virtual Private Cloud

Q15. What is a VPC?

Answer: A **VPC (Virtual Private Cloud)** is a **logically isolated virtual network** in AWS where you launch resources. You control the **IP address range (CIDR)**, **subnets**, **route tables**, and **gateways** — it's your own private slice of the AWS network.

Q16. What is the difference between a public and a private subnet?

Answer: A **public subnet** has a route to an **Internet Gateway**, so its resources can be reached from/access the internet. A **private subnet** has **no direct route to the internet**; its resources stay internal (often using a NAT for outbound-only access).

Q17. What is an Internet Gateway (IGW)?

Answer: An **Internet Gateway** is a VPC component that **allows communication between resources in your VPC and the internet**. A subnet becomes public by having a route table entry pointing internet-bound traffic to the IGW.

Q18. What is a NAT Gateway and why is it used?

Answer: A **NAT (Network Address Translation) Gateway** lets instances in a **private subnet initiate outbound internet connections** (e.g. to download updates) while **preventing inbound connections** from the internet. It sits in a public subnet and provides one-way outbound access.

Q19. What is CIDR notation?

Answer: **CIDR (Classless Inter-Domain Routing)** notation defines an IP range, e.g. `10.0.0.0/16`. The number after the slash is the **network prefix length**; a smaller number means a larger range. `/16` gives ~65,536 addresses, `/24` gives 256.

Q20. What is a route table?

Answer: A **route table** contains **rules (routes)** that determine where network traffic from a subnet is directed — e.g. send `0.0.0.0/0` (internet) to an Internet Gateway, or local VPC traffic internally. Each subnet is associated with exactly one route table.

Q21. What is VPC Peering?

Answer: **VPC Peering** is a networking connection between **two VPCs** that lets them **communicate privately** using internal IPs as if on the same network. It is **non-transitive** (if A peers with B and B with C, A cannot reach C through B).

Q22. What is a VPC Endpoint?

Answer: A **VPC Endpoint** lets resources in your VPC **privately connect to AWS services** (like S3 or DynamoDB) **without going over the public internet**, improving security and reducing data transfer cost. Types include **Gateway endpoints** and **Interface endpoints**.

S3 — Simple Storage Service

Q23. What is Amazon S3?

Answer: **S3 (Simple Storage Service)** is a highly durable, scalable **object storage** service. You store **objects (files + metadata)** in **buckets**. It offers **11 nines (99.999999999%) of durability** and is ideal for backups, static websites, data lakes, and media.

Q24. What is the difference between object storage and block storage?

Answer: **Object storage (S3)** stores data as whole **objects** with metadata in a flat namespace, accessed via API/HTTP — great for unstructured data at scale. **Block storage (EBS)** stores data in fixed-size **blocks** like a disk attached to a server, ideal for OS and databases that need low-latency read/write.

Q25. What are S3 storage classes?

Answer: Storage classes optimize cost vs. access needs: **S3 Standard** (frequent access), **Standard-IA / One Zone-IA** (infrequent access), **Intelligent-Tiering** (auto-moves data), and **Glacier / Glacier Deep Archive** (cheap, long-term archival with retrieval delays).

Q26. How can you secure data in S3?

Answer: Use **IAM policies and bucket policies** for access control, **block public access** settings, **encryption at rest** (SSE-S3, SSE-KMS) and **in transit (HTTPS)**, **versioning** to recover from deletes, and **MFA Delete** for extra protection.

Q27. What is S3 versioning?

Answer: **Versioning** keeps **multiple versions of an object** in a bucket, so you can **recover from accidental overwrites or deletions**. When enabled, a delete adds a delete marker rather than removing the data permanently.

Q28. What is an S3 lifecycle policy?

Answer: A **lifecycle policy** automatically **transitions objects between storage classes** (e.g. Standard → Glacier after 30 days) or **expires/deletes** them after a set time, helping reduce storage costs without manual effort.

Q29. What is the difference between S3 and EBS/EFS?

Answer: **S3** is object storage accessed via API, not a filesystem. **EBS** is block storage attached to a **single EC2 instance** in one AZ. **EFS (Elastic File System)** is a **shared file system** that multiple EC2 instances can mount concurrently across AZs.

IAM — Identity & Access Management

Q30. What is AWS IAM?

Answer: **IAM (Identity and Access Management)** controls **who is authenticated and authorized** to use AWS resources. It manages **users, groups, roles, and policies** to grant fine-grained, least-privilege permissions.

Q31. What is the difference between an IAM user and an IAM role?

Answer: An **IAM user** is a **permanent identity** with long-term credentials for a person or application. An **IAM role** has **no permanent credentials** and is **assumed temporarily** to gain a set of permissions — ideal for EC2 instances, services, or cross-account access.

Q32. What is an IAM policy?

Answer: An **IAM policy** is a **JSON document** that defines **permissions** — which **actions** are allowed/denied on which **resources**, under optional **conditions**. Policies are attached to users, groups, or roles.

Q33. What is the principle of least privilege?

Answer: It means granting **only the minimum permissions necessary** to perform a task, and nothing more. This reduces the security risk if credentials are compromised. It is a core IAM best practice.

Q34. Why should you use IAM roles for EC2 instead of storing access keys?

Answer: Attaching an **IAM role** to an EC2 instance provides **temporary, automatically-rotated credentials** to applications, so you **never hard-code long-term access keys** on the server. This is far more secure and easier to manage.

Q35. What is MFA and why is it recommended?

Answer: **MFA (Multi-Factor Authentication)** adds a **second verification factor** (like a code from a device) on top of a password. It's strongly recommended — especially for the **root account** and privileged users — because it protects access even if the password is stolen.

RDS — Relational Database Service

Q36. What is Amazon RDS?

Answer: **RDS (Relational Database Service)** is a **managed relational database** service that handles provisioning, patching, backups, and scaling. It supports engines like **MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Amazon Aurora**.

Q37. What is a Multi-AZ deployment in RDS?

Answer: **Multi-AZ** maintains a **synchronous standby replica in a different AZ**. If the primary fails, RDS **automatically fails over** to the standby. It is for **high availability and durability**, not for scaling reads.

Q38. What is the difference between Multi-AZ and Read Replicas?

Answer: **Multi-AZ** is for **high availability/failover** (synchronous standby, not used for reads). **Read Replicas** are for **scaling read traffic** (asynchronous copies you can read from). They solve different problems and can be used together.

Q39. What is Amazon Aurora?

Answer: **Aurora** is AWS's **cloud-native relational database**, compatible with MySQL and PostgreSQL, offering **up to several times better performance**, automatic storage scaling, and **fault-tolerant, self-healing storage** replicated across multiple AZs.

Q40. When would you choose DynamoDB over RDS?

Answer: Choose **DynamoDB** (a managed **NoSQL key-value/document** database) when you need **massive scale, single-digit millisecond latency, flexible schema**, and high throughput. Choose **RDS** when you need **relational data, complex queries, joins, and ACID transactions** with a fixed schema.

EKS & Containers

Q41. What is Amazon EKS?

Answer: **EKS (Elastic Kubernetes Service)** is a **managed Kubernetes service**. AWS runs and maintains the **Kubernetes control plane** (API server, etcd, scheduler) across multiple AZs, while you run your applications on **worker nodes**. It lets you use standard Kubernetes without managing the control plane yourself.

Q42. What parts of EKS does AWS manage vs. the customer?

Answer: AWS **manages the control plane** — the API server, etcd, scheduler, and controller manager — keeping it **highly available, patched, and scaled**. The customer manages the **data plane**: the **worker nodes** (EC2 or Fargate), the workloads/Pods, networking config, and scaling (unless using Fargate, which abstracts the nodes).

Q43. What is the difference between EKS, ECS, and Fargate?

Answer: **ECS (Elastic Container Service)** is AWS's **proprietary** container orchestrator. **EKS** runs **standard Kubernetes**. **Fargate** is a **serverless compute engine** for containers — it runs your containers **without you managing EC2 worker nodes**, and works as a launch type with both ECS and EKS.

Q44. What is a Managed Node Group in EKS?

Answer: A **Managed Node Group** automates the **provisioning and lifecycle of EC2 worker nodes** for an EKS cluster — including creation, updates, and graceful termination during scaling — so you don't manually manage the underlying instances.

Q45. What is Amazon ECR?

Answer: **ECR (Elastic Container Registry)** is a **managed Docker/OCI container image registry**. You push images to ECR and pull them into EKS/ECS. It integrates with IAM for access control and supports image scanning.

Q46. How do worker nodes in EKS communicate with the control plane?

Answer: Each worker node runs the **kubelet** and **kube-proxy**; the kubelet **registers with and communicates with the EKS-managed API server endpoint**. EKS uses the **Amazon VPC CNI plugin** to give Pods VPC IP addresses, and IAM (via IAM Roles for Service Accounts) for fine-grained permissions.

Q47. How do you typically deploy an application to EKS in a DevOps pipeline?

Answer: Build a Docker image and push it to **ECR**, then apply Kubernetes manifests (Deployments, Services) to the cluster using **kubectl** or tools like **Helm**, often automated through **CodePipeline/Jenkins**. **eksctl** is commonly used to create/manage the cluster itself.

DevOps, IaC & Monitoring

Q48. What is Amazon CloudWatch?

Answer: **CloudWatch** is AWS's **monitoring and observability** service. It collects **metrics, logs, and events**, lets you build **dashboards and alarms**, and can trigger actions (like Auto Scaling or notifications) when thresholds are crossed.

Q49. What is AWS CloudFormation and how does it relate to IaC?

Answer: **CloudFormation** is AWS's **Infrastructure as Code (IaC)** service. You define resources in a **JSON/YAML template**, and CloudFormation **provisions and manages them as a stack** — making infrastructure **repeatable, version-controlled, and automated**. (Terraform is a popular multi-cloud alternative.)

Q50. What is the difference between CodeCommit, CodeBuild, CodeDeploy, and CodePipeline?

Answer: **CodeCommit** is a managed Git source repository. **CodeBuild** compiles code and runs tests. **CodeDeploy** automates deployments to compute targets. **CodePipeline** is the **orchestrator** that ties these stages into an automated **CI/CD pipeline**.
